7th Week CODES

Q1. Extensible Notification System

===

```
public interface INotificationSender

{

   Task SendAsync(string msg);

}


public class EmailSender : INotificationSender

{

   public async Task SendAsync(string msg)

   {

      await Task.Delay(100);

      Console.WriteLine("Email Sent");

   }

}


public class SmsSender : INotificationSender

{

   public async Task SendAsync(string msg)

   {

      await Task.Delay(100);

      Console.WriteLine("SMS Sent");

   }

}


public class NotificationService

{
```

```
    private readonly IEnumerable<INotificationSender> _senders;

    public NotificationService(IEnumerable<INotificationSender> senders)
    {
        _senders = senders;
    }

    public async Task NotifyAll(string msg)
    {
        await Task.WhenAll(_senders.Select(s => Retry(s, msg)));
    }

    private async Task Retry(INotificationSender s, string msg)
    {
        try { await s.SendAsync(msg); }
        catch
        {
            try { await s.SendAsync(msg); }
            catch { Console.WriteLine("Failed"); }
        }
    }
}
```

==

Q2. Thread-Safe In-Memory Cache

```
public class CacheItem
{
    public object Value;
```

```
    public DateTime Expiry;
}

public class CacheService
{
    private static readonly CacheService _instance = new();
    private ConcurrentDictionary<string, CacheItem> _cache = new();

    public static CacheService Instance => _instance;

    public void Set(string key, object value, int seconds)
    {
        _cache[key] = new CacheItem
        {
            Value = value,
            Expiry = DateTime.Now.AddSeconds(seconds)
        };
    }

    public object Get(string key)
    {
        if (_cache.TryGetValue(key, out var item))
        {
            if (item.Expiry > DateTime.Now)
                return item.Value;
            _cache.TryRemove(key, out _);
        }
        return null;
    }
```

```
}
```

Q3. JWT Auth with Random 401

```
builder.Services.AddAuthentication("Bearer")
.AddJwtBearer("Bearer", opt =>
{
    opt.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ClockSkew = TimeSpan.FromMinutes(2)
    };
});

app.UseAuthentication();
app.UseAuthorization();
```

==

Q4. Rate Limiting

```
public class RateLimiter
{
    private ConcurrentDictionary<string, (int count, DateTime start)> map = new();

    public bool Allow(string user)
```

```
    {
        var now = DateTime.Now;
        var data = map.GetOrAdd(user, (0, now));

        if ((now - data.start).TotalSeconds > 10)
            map[user] = (1, now);
        else if (data.count >= 5)
            return false;
        else
            map[user] = (data.count + 1, data.start);

        return true;
    }
}
```

==

Q5. Deadlock & Fix

```
object A = new();
object B = new();

// Deadlock
lock(A)
{
    lock(B) { }
}

if (Monitor.TryEnter(A, 1000))
{
```

```
    if (Monitor.TryEnter(B, 1000))
    {

    }
}
```

===

Q6. Custom HashMap

```
class MyHashMap<K,V>
{
    List<(K key,V val)>[] buckets = new List<(K,V)>[10];

    int Hash(K key) => key.GetHashCode() % buckets.Length;

    public void Add(K k, V v)
    {
        int i = Hash(k);
        if (buckets[i] == null) buckets[i] = new();
        buckets[i].Add((k,v));
    }

    public V Get(K k)
    {
        int i = Hash(k);
        foreach (var p in buckets[i])
            if (p.key.Equals(k)) return p.val;
        return default;
    }
```

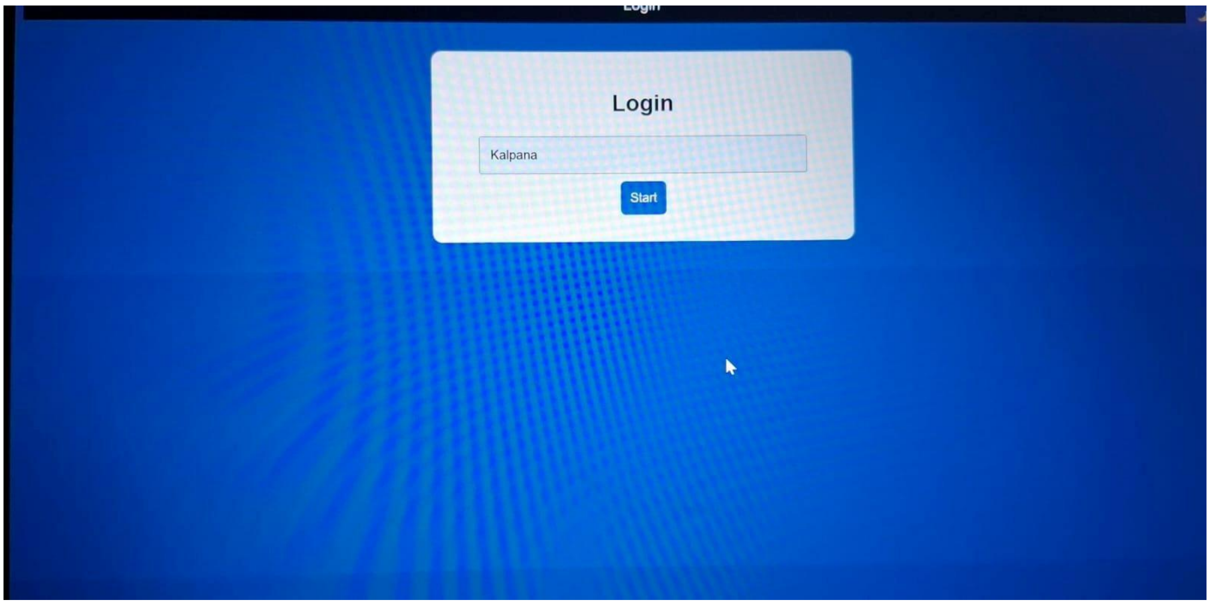
}

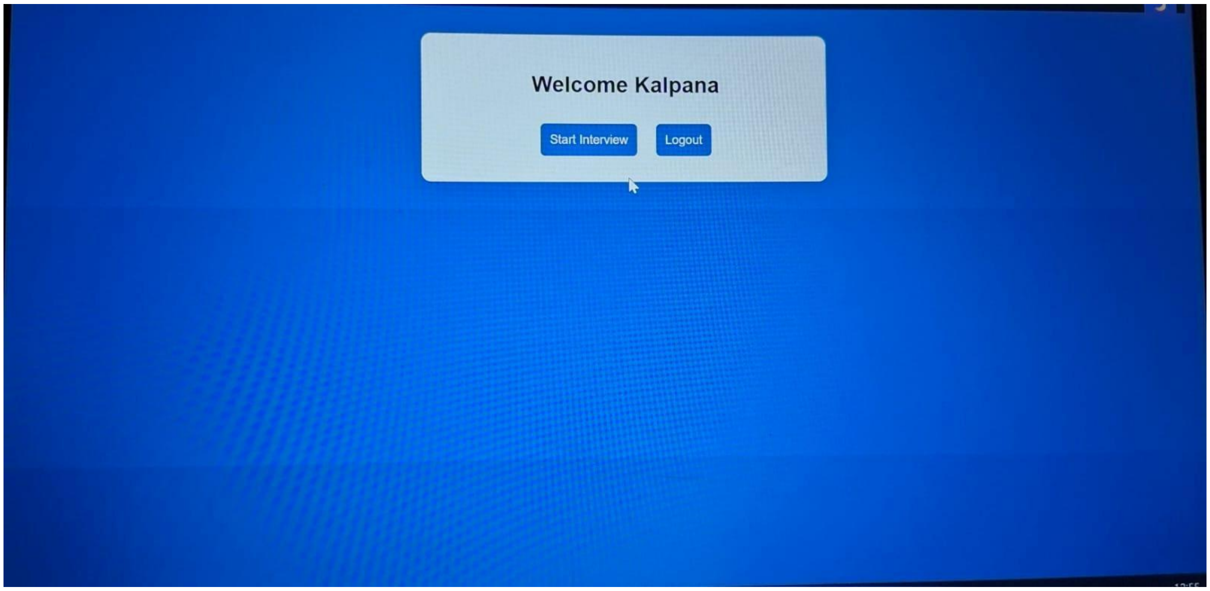
===

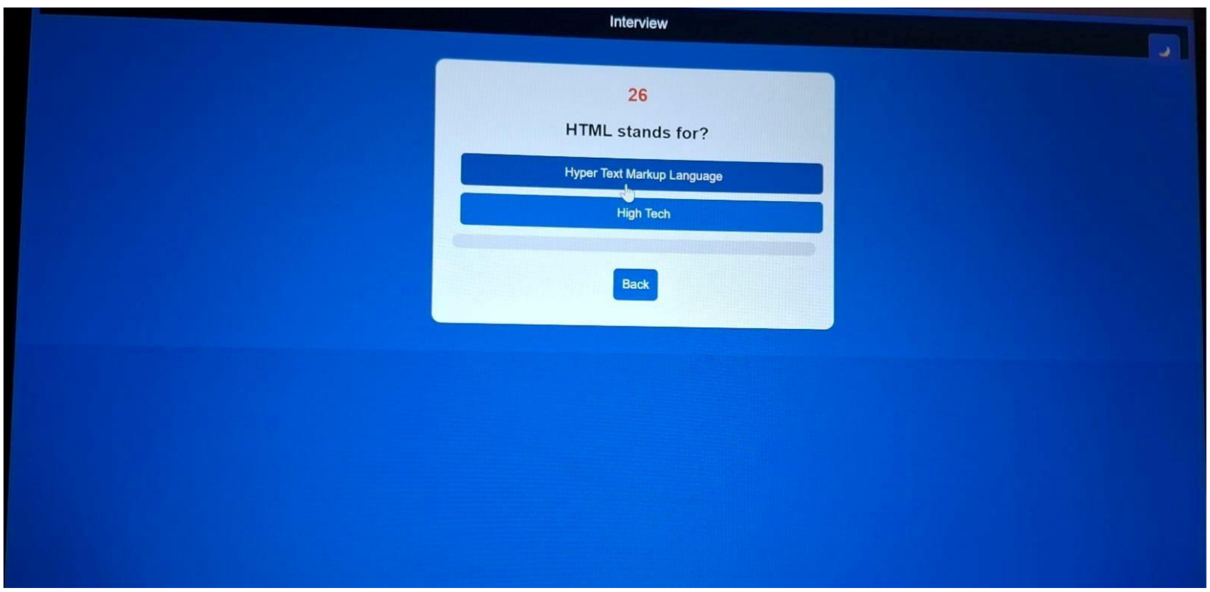
Q7. High-Load Order Processing System

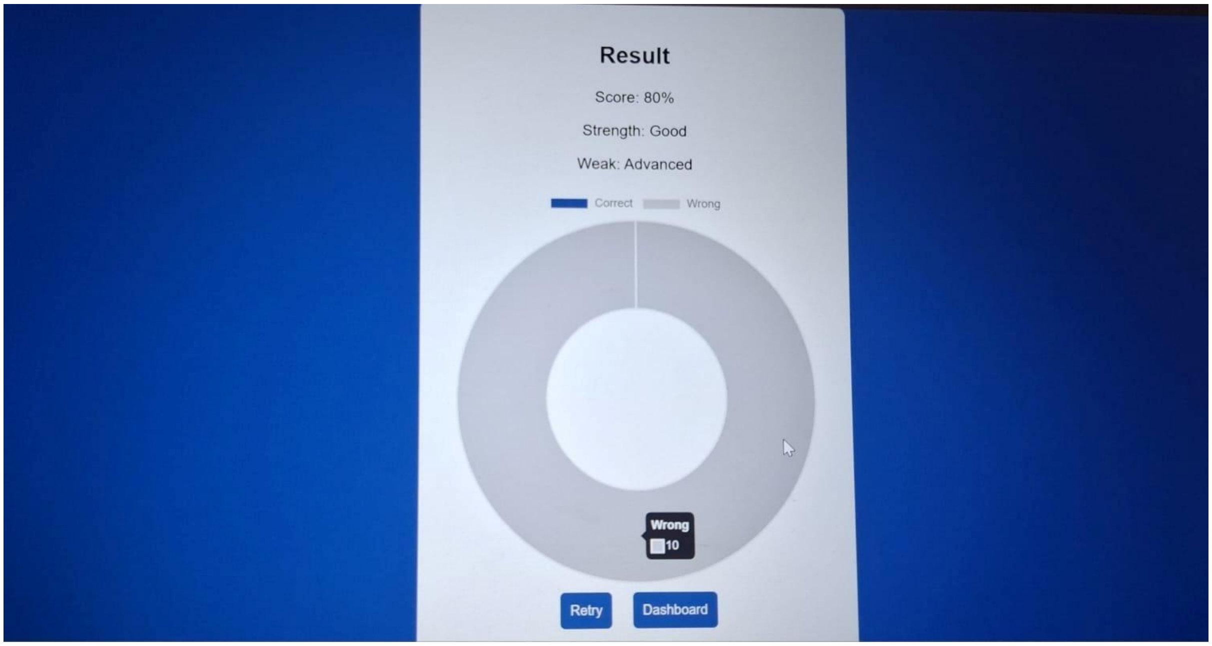
```
public class OrderService
{
    public async Task PlaceOrder()
    {

        Save order (DB)
        Cache order
        // Send notification async
    }
}
```

Interview Simulator 1st Mini project otputs :

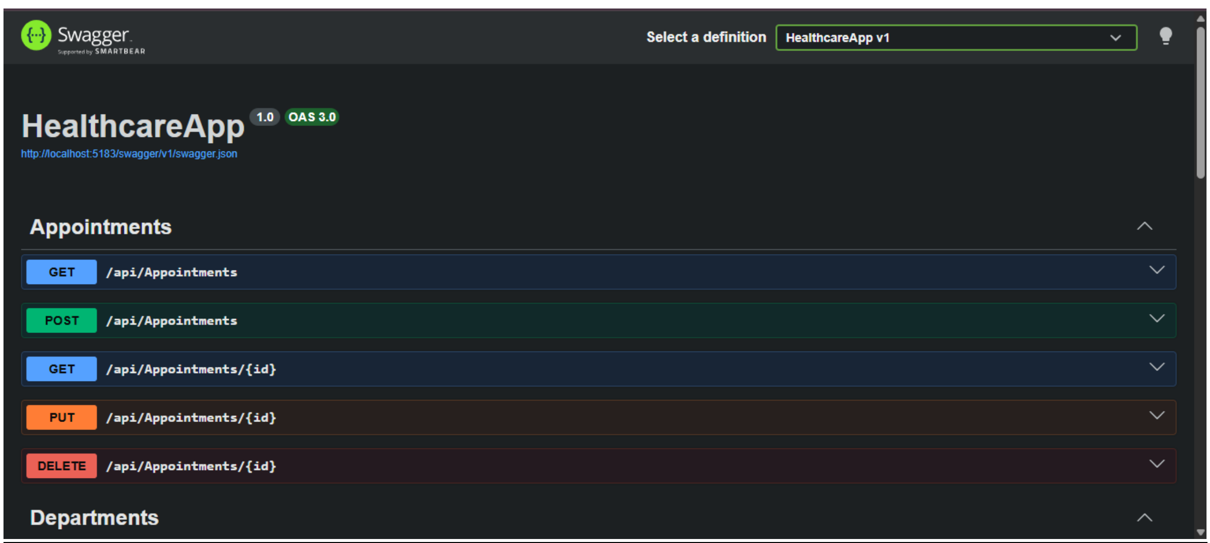



Welcome Kalpana
Start Interview
Logout

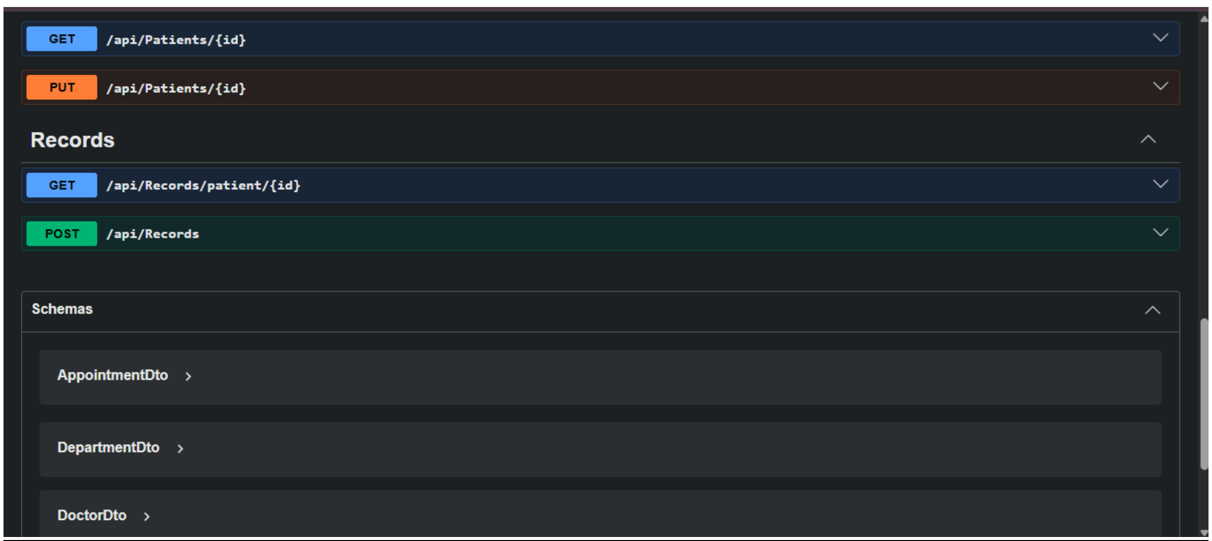

Interview
26
HTML stands for?
Hyper Text Markup Language
High Tech
Back



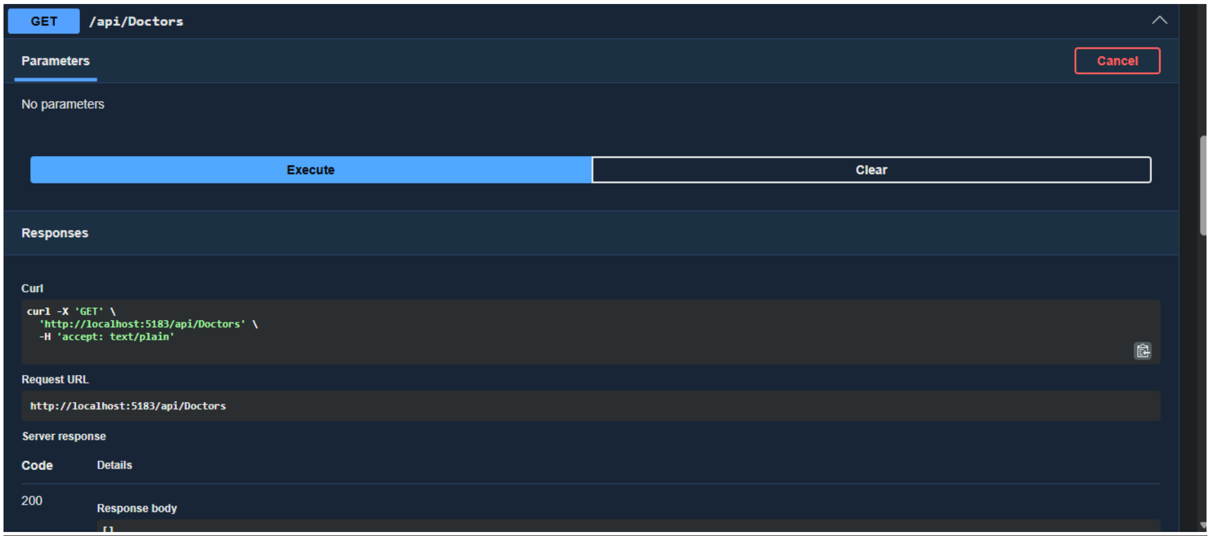
2nd Mini Project Outputs : Health Care Project –

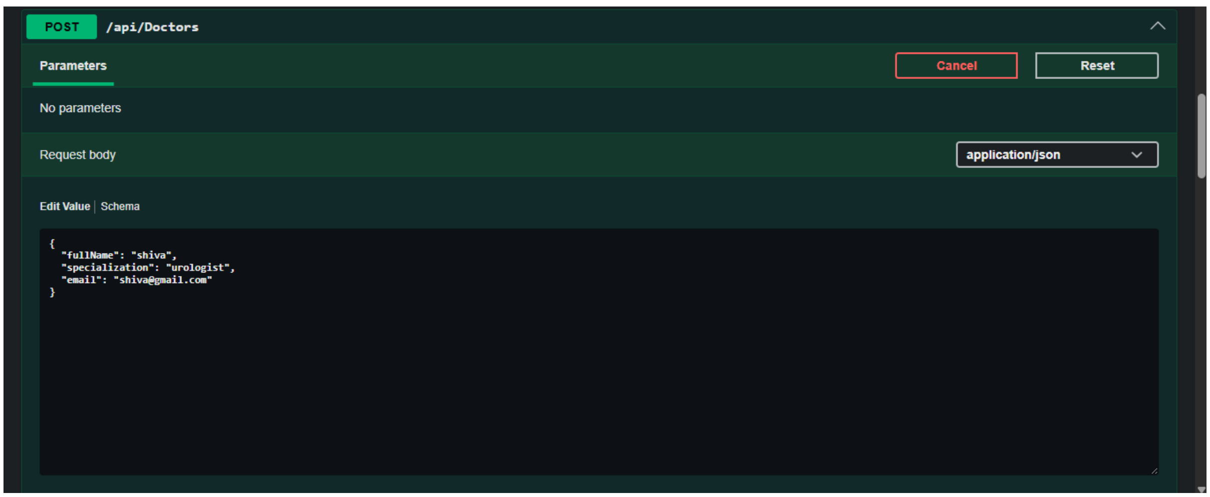


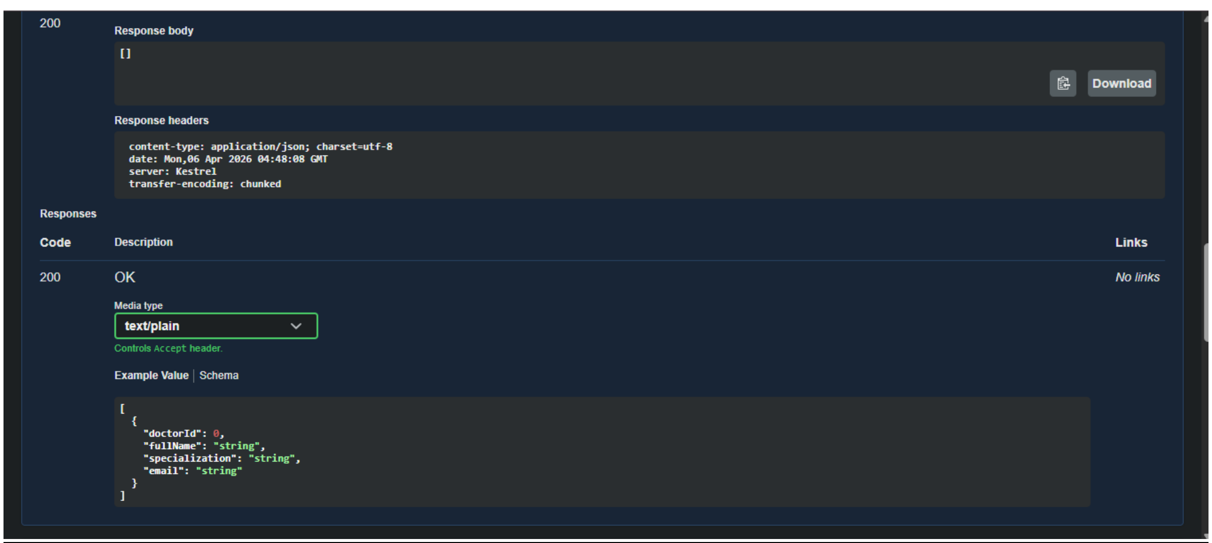

Departments
GET /api/Departments
POST /api/Departments
Doctors
GET /api/Doctors
POST /api/Doctors
GET /api/Doctors/{id}
PUT /api/Doctors/{id}
DELETE /api/Doctors/{id}
Patients
POST /api/Patients/register

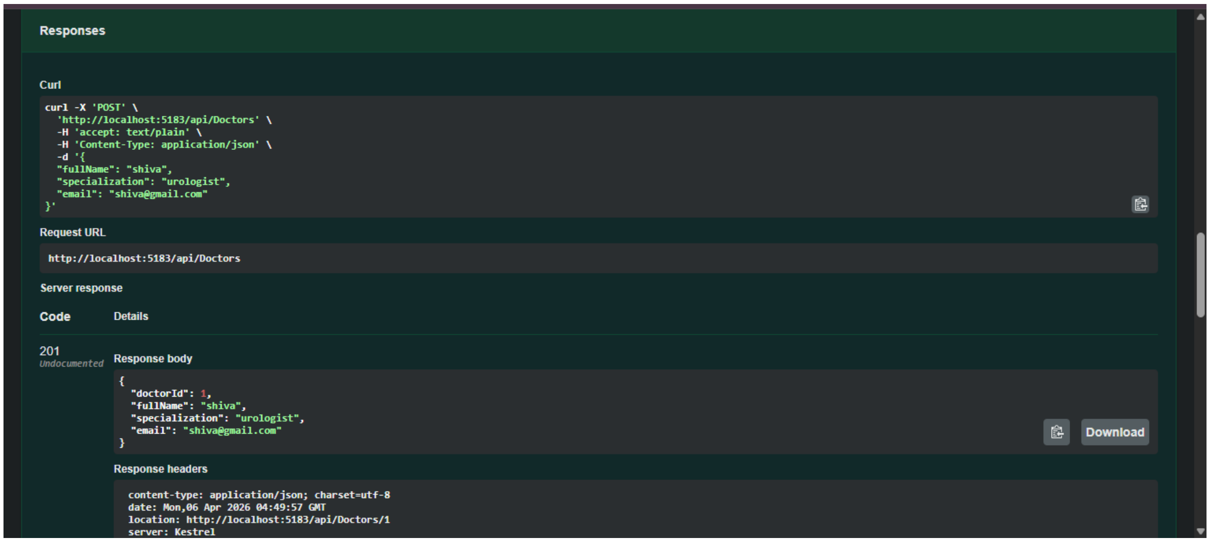

GET /api/Patients/{id}
PUT /api/Patients/{id}
Records
GET /api/Records/patient/{id}
POST /api/Records
Schemas
AppointmentDto
DepartmentDto
DoctorDto


POST /api/Records
Schemas
AppointmentDto
DepartmentDto
DoctorDto
MedicalRecordDto
PatientDto


GET
/api/Doctors
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:5183/api/Doctors' \
-H 'accept: text/plain'
Request URL
http://localhost:5183/api/Doctors
Server response
Code
Details
200
Response body


POST
/api/Doctors
Parameters
Cancel
Reset
No parameters
Request body
application/json
Edit Value | Schema
{
"fullName": "shiva",
"specialization": "urologist",
"email": "shiva@gmail.com"
}


200
Response body
[]
Download
Response headers
content-type: application/json; charset=utf-8
date: Mon,06 Apr 2026 04:48:08 GMT
server: Kestrel
transfer-encoding: chunked
Responses
Code
Description
Links
200
OK
No links
Media type
text/plain
Controls Accept header.
Example Value | Schema
[
{
"doctorId": 0,
"fullName": "string",
"specialization": "string",
"email": "string"
}
]


Responses
Curl
curl -X 'POST' \
'http://localhost:5183/api/Doctors' \
-H 'accept: text/plain' \
-H 'Content-Type: application/json' \
-d '{
"fullName": "shiva",
"specialization": "urologist",
"email": "shiva@gmail.com"
}'
Request URL
http://localhost:5183/api/Doctors
Server response
Code
Details
201
Undocumented
Response body
{
"doctorId": 1,
"fullName": "shiva",
"specialization": "urologist",
"email": "shiva@gmail.com"
}
Download
Response headers
content-type: application/json; charset=utf-8
date: Mon,06 Apr 2026 04:49:57 GMT
location: http://localhost:5183/api/Doctors/1
server: Kestrel